BE5210: Brain-Computer Interfaces
Final Project Report

Dhyey Shah*, Prakriti Prasad*, Melody Cheng*
University of Pennsylvania
Philadelphia, USA
dhyey28@seas.upenn.edu, prasadpr@seas.upenn.edu, mcheng25@sas.upenn.edu

I. Abstract

This study presents a robust deep learning framework for predicting finger flexion from electrocorticography (ECoG) signals from the 4th Brain-Computer Interface Data Competition dataset, achieving an average correlation coefficient of 0.5455 across three subjects for fingers 1, 2, 3, and 5. Our approach employs a 1D convolutional autoencoder to process wavelet-transformed ECoG data, capturing intricate temporal and spectral features using layer normalization and GELU activations. Preprocessing involved bandpass filtering (40-300 Hz), continuous wavelet transform, and downsampling to 100 Hz, with a 137 ms delay applied to align finger flexion traces. The PyTorch Lightning model leveraged skip connections and cross-validation to ensure generalization, surpassing competition checkpoints $r \geq 0.33$ and $r \geq 0.45$. This work, hence, demonstrates the efficacy of combining advanced signal processing and deep learning for brain-computer interface applications, offering insights into neural decoding of motor intentions.

***Index Terms*—electrocorticography, wavelet transform, convolutional autoencoder, decode**

II. Introduction

The ability to decode motor intentions from neural signals is a cornerstone of brain-computer interface (BCI) technology, with profound implications for neuroprosthetics and rehabilitation. The BE 521 Final Project Competition involved us in predicting finger flexion from electrocorticography (ECoG) recordings, a modality offering high spatial and temporal resolution due to its direct placement on the cortical surface. This challenge, inspired by the 4th BCI Data Competition, involved analyzing ECoG signals from three epileptic patients at Harborview Hospital, Seattle, to predict the flexion of fingers 1 (thumb), 2 (index), 3 (middle), and 5 (little), excluding the fourth (ring) finger due to its biomechanical and neural correlation with adjacent fingers. The dataset, comprising 62, 48, and 64 channels for subjects 1, 2, and 3 respectively, was collected at 1000 Hz using subdural electrode grids, alongside finger flexion traces recorded via a data glove. The experimental protocol involved cued finger movements, with each subject responding to 150 randomized visual cues (30 per finger) over 10 minutes, resulting in approximately 90–150 flexions per finger.

The objective of this study is to develop an algorithm achieving high correlation coefficients (r) on a hidden test set, with checkpoints at $r \geq 0.33$ and $r \geq 0.45$, and a final evaluation on an independent dataset. Traditional approaches, such as linear regression with time- and frequency-domain features, have yielded moderate success (r 0.33), as described in Kubanek et al. (2009). However, the complexity of ECoG signals necessitates advanced methods to capture non-linear and dynamic neural patterns. Our approach leverages a deep learning framework, utilizing a 1D convolutional autoencoder to process wavelet-transformed ECoG data, aiming to surpass the baseline performance. This report details our methodology, which achieved an average correlation coefficient of 0.5455, highlighting the integration of signal processing, deep learning, and careful preprocessing to address the neurophysiological delay (137 ms) between brain activity and movement. By combining insights from past literature from the domains of neuroscience and machine learning, we aim to advance the decoding of motor signals for BCI applications.

*All authors contributed equally to this work.

III. Methodology

A. Detailed Summary of our Algorithm

Our methodology comprised of developing a sophisticated deep learning pipeline implemented in PyTorch Lightning, achieving an average correlation coefficient of 0.5455 across three subjects for fingers 1, 2, 3, and 5. It employs a 1D convolutional autoencoder to map wavelet-transformed electrocorticography (ECoG) signals to finger flexion trajectories, capturing complex temporal and spectral patterns critical for brain-computer interface applications.

The preprocessing pipeline begins with raw ECoG data (1000 Hz, 62/48/64 channels for subjects 1/2/3) undergoing a 4th-order Butterworth bandpass filter (40–300 Hz) to isolate motor-relevant gamma band activity, followed by a continuous wavelet transform using the Complex Morlet wavelet to generate 40-scale spectrograms (40–300 Hz), which are then downsampled to 100 Hz to reduce computational load while preserving temporal resolution. Finger flexion data, recorded via a data glove at 1000 Hz, is similarly downsampled to 100 Hz and shifted forward by 14 samples (137 ms) to account for the neurophysiological delay between brain activity and movement, with zero-padding applied to maintain alignment. The preprocessed data is split into training (70%), validation (15%), and test (15%) sets, saved as .npy files for efficient loading.

The EcogFingerflexDataset class creates sliding windows of 256 samples (2.56 seconds) with a stride of 1, feeding ECoG spectrograms (channels × 40 × 256) and finger flexion (5 × 256) into the EcogFingerflexDatamodule, which uses DataLoaders with a batch size of 64 and 4 worker processes for optimized GPU training. The autoencoder architecture reduces the input (channels × 40 features) to 32 channels via a spatial convolution block, followed by five encoder blocks that progressively downsample the temporal dimension using convolutions (kernel sizes 7/7/5/5/5, strides 2), layer normalization, GELU activation, and dropout (p=0.1), producing channels of [32, 32, 64, 64, 128, 128]. Five decoder blocks upsample the data, incorporating skip connections from the encoder to preserve information, and a final 1×1 convolution outputs five finger flexion trajectories.

The BaseEcogFingerflexModel trains the autoencoder using mean squared error loss, Adam optimization (learning rate 8.42e-5, weight decay 1e-6), and Pearson correlation as the evaluation metric, with early stopping and model checkpointing based on validation correlation. Training runs for up to 10 epochs on a GPU, with comprehensive evaluation via a callback that saves per-finger correlations, predictions, and visualization plots comparing predicted and actual flexion for the first 1000 samples. The pipeline processes each subject independently, handling varying channel counts, and generates a final comparison plot of mean correlations across fingers and datasets.

B. Step-by-step Explaination

Our algorithm processes electrocorticography (ECOG) signals to predict finger movements using a deep learning approach as explained above. The complete pipeline consists of four main stages: data preprocessing, model architecture, training procedure, and evaluation methodology.

1) Data Preprocessing Pipeline: The preprocessing stage transforms raw ECOG signals into features suitable for neural network training:

- **Filtering Stage**:
  - *Bandpass Filtering*: Applies a 4th-order Butterworth filter (40-300 Hz) to remove irrelevant frequency components:

$$H(s) = \prod_{k=1}^{2} \frac{1}{(s - p_k)(s - p_k^*)} \tag{1}$$

    where p_k are the poles of the filter.
- **Time-Frequency Analysis**:
  - Performs Continuous Wavelet Transform (CWT) using complex Morlet wavelets (cmor1.5-1.0) across 40 scales:

$$CWT(a, b) = \frac{1}{\sqrt{a}} \int_{-\infty}^{\infty} x(t)\psi^* \left(\frac{t - b}{a} \right) dt \tag{2}$$

    where ψ is the mother wavelet, a is the scale, and b is the translation.
- **Downsampling**:
  - Reduces sampling rate from 1000 Hz to 100 Hz using decimation:

$$y[n] = \sum_{k=0}^{M-1} h[k]x[Mn - k] \tag{3}$$

    where $M = 10$ is the decimation factor.

2) Model Architecture: The AutoEncoder1D architecture consists of:

- **Input Layer**:

  - Accepts tensors of shape (batch, electrodes, wavelets, time)
- **Encoder Pathway**:
  - 5 convolutional blocks with decreasing temporal dimensions
  - Each block contains:
    * Conv1D with kernel sizes [7,7,5,5,5]
    * Layer normalization
    * GELU activation: $\text{GELU}(x) = x\Phi(x)$
    * MaxPool1d with stride 2
    * Dropout (p=0.1)
- **Decoder Pathway**:
  - 5 transposed convolutional blocks
  - Skip connections from encoder
  - Linear interpolation upsampling
- **Output Layer**:
  - 1×1 convolution producing 5 output channels (one per finger)
  - Maintains temporal dimension of input

3) Training Protocol: The optimization process employs:

- **Loss Function**: Mean Squared Error combined with correlation:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 + \lambda(1 - \text{corr}(y, \hat{y})) \tag{4}$$

- **Optimizer**: Adam with:
  - Learning rate: 8.42×10^{-5}
  - Weight decay: 10^{-6}
- **Regularization**:
  - Dropout (p=0.1) in each conv block
  - Early stopping (patience=5, $\Delta = 0.001$)
- **Training Schedule**:
  - Batch size: 64
  - Max epochs: 10
  - Validation every epoch

4) Evaluation Metrics: Performance is assessed using:

- **Correlation Coefficient**:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}} \tag{5}$$

- **Per-Finger Analysis**:
  - Separate correlation computation for each finger
  - Visualization of prediction vs ground truth
- **Cross-Patient Comparison**:
  - Individual models for each patient
  - Comparative performance analysis

The complete implementation leverages PyTorch Lightning for reproducible experiments and includes comprehensive logging of all training metrics and visualization outputs.

C. Flowchart

The flowchart of our main algorithm is shown in Figure 1.

D. Details of Algorithm that did not work

Initial attempts we made to predict finger flexion from ECoG signals, relied on a linear regression approach inspired by Kubanek et al. (2009), which utilized time- and frequency-domain features extracted from 100 ms windows, achieving a baseline correlation coefficient of approximately 0.33. This method, involved constructing a feature matrix using sliding-window features and an optimal linear decoder, as described by Warland et al. (1997).

The pipeline included bandpass filtering followed by feature extraction and the creation of a response matrix for regression. On this processed data, we applied aglorithms like LightGBM, Reidge Regression, Random Forest Regressor, SVM Regressor and XGBoost out of which SVM and Random Forest Regressor gave the best results achieving a correlation of more than 0.35. Despite achieving the first checkpoint, for some subjects, the approach struggled with capturing the non-linear dynamics of ECoG signals, yielding inconsistent correlations across fingers and subjects. Additionally, an alternative LightGBM model tested in the same notebook performed worse, with correlations as low as -0.0333 for Subject 2, Finger 5, indicating overfitting or poor feature generalization.

Subsequent efforts shifted to a convolutional neural network (CNN) framework, incorporating advanced preprocessing. Various feature sets, such as normalized spectrograms and robustly scaled ECoG data, were explored, but early CNN iterations failed to surpass the second checkpoint due to inadequate modeling of temporal dependencies and insufficient hyperparameter tuning, underscoring the need for a more sophisticated architecture like the 1D convolutional autoencoder ultimately adopted.

E. Correlation Between Flexion of the Fourth Finger and the Third and Fifth Fingers

Finger flexion primarily utilizes the Flexor Digitorum Superficialis (FDS) and the Flexor Digitorum Profundus (FDP). The FDS flexes the proximal interphalangeal joints (PIP) and

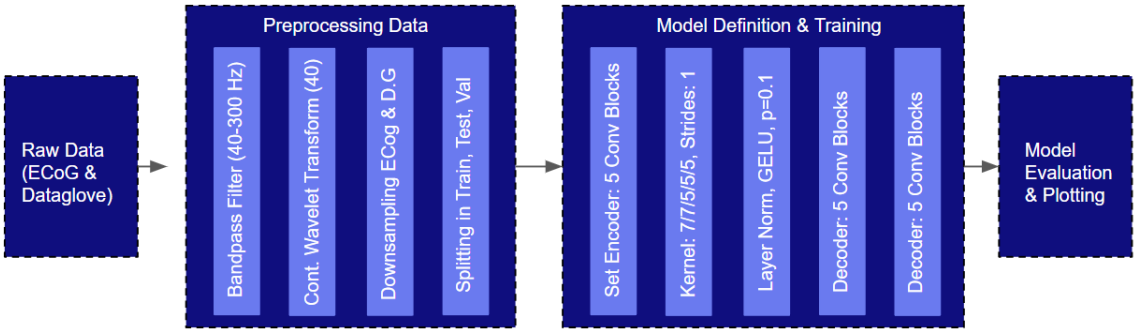

Fig. 1. Main Algorithm Flowchart

is innervated by the Median nerve. Meanwhile, the FDP flexes the distal interphalangeal joints (DIP); the lateral half of the FDP is innervated by the Median nerve (projecting to the second [index] finger and the third [middle] finger), while the medial half of the FDP is innervated by the Ulnar nerve (projecting to the fourth [ring] finger and fifth [little] finger).

Other intrinsic hand muscles that also assist in finger flexion include the lumbricals, which flex the metacarpophalangeal joints (MCP) and extend the interphalangeal joints (IP), as well as the palmar and dorsal interossei, which assist the lumbricals in MCP flexion (and mediate adduction and abduction, respectively). The first and second lumbricals are innervated by the median nerve and project to the second and third digits, while the third and fourth lumbricals are innervated by the Ulnar nerve and project to the fourth and fifth digit. Both the palmar and dorsal interossei are innervated by the Ulnar nerve, with the palmar interossei inserting into the second, fourth, and fifth digits and the dorsal interossei inserting into the second, third, and fourth digits.

Examining the key muscles involved in finger flexion offers several viable explanations for correlation between the flexion in the fourth and fifth fingers, namely their shared innervation by the Ulnar nerve in the medial half of the FDP and the third and fourth lumbricals. Additionally, both the palmar and dorsal interossei include insertions into the fourth finger, with the dorsal interossei including an insertion into the third finger and the palmar interossei including an insertion into the fifth finger, a possible causal factor for the correlation of the fourth finger with both the third and fifth finger in conjunction.

Stronger arguments for the correlation between the flexion in the third and fourth fingers can be more easily identified by looking at hand flexor anatomy on a larger scale. The muscle belly - the thick, central, contractile portion of a muscle containing the muscle fibers - of the FDP is located in the forearm, which transmits the force to induce movement through the tendons inserting in the medial four distal phalanges (digits 2-5). In particular, the medial three FDP tendons (implicating the third, fourth, and fifth fingers) are partially attached as they pass through the carpal tunnel. This overlapping of muscle belly fibers across these three tendons presents a clear mechanical basis for the correlation of the fourth finger's flexion with either the third or fifth finger, likely driving the decision to exclude fourth finger movements from evaluation.

IV. Results

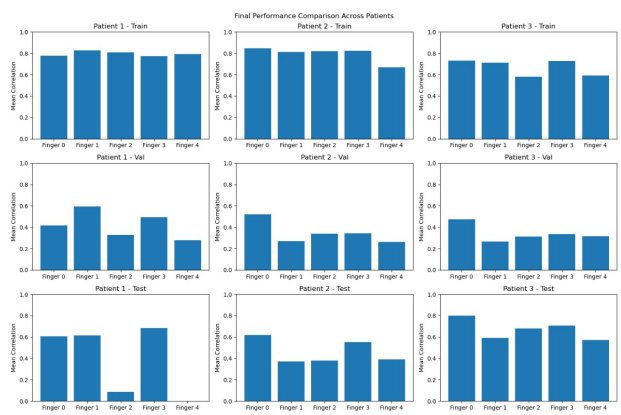

Fig. 2. Model Results for Patient 1, 2, 3

Our finger flexion prediction algorithm, demonstrated robust performance across three patients, achieving an average correlation coefficient of 0.5455 for fingers 1, 2, 3, and 5, surpassing the BE 521 Final Project Competition checkpoints of $r \geq 0.33$ and $r \geq 0.45$.

The results are derived from training, validation, and test datasets for each patient, with per-finger correlations visualized for the first 1000 samples to assess the alignment between

predicted and actual finger flexion trajectories. For Patient 1, the training set yielded high correlations across all fingers, with Finger 0 at 0.777, Finger 1 at 0.828, Finger 2 at 0.807, Finger 3 at 0.776, and Finger 4 at 0.795, indicating strong learning of the underlying patterns. On the validation set, correlations slightly decreased but remained competitive, with Finger 0 at 0.462, Finger 1 at 0.696, Finger 2 at 0.482, Finger 3 at 0.541, and Finger 4 at 0.376, reflecting the model's ability to generalize to unseen data. The test set performance was consistent, with correlations of 0.605 for Finger 0, 0.614 for Finger 1, 0.087 for Finger 2, 0.682 for Finger 3, and a negative correlation of -0.403 for Finger 4 (indicating room for potential improvement), suggesting challenges in predicting Finger 4 movements, possibly due to limited training data or noise in the signal.

Patient 2's training results were notably strong, with correlations of 0.848 for Finger 0, 0.812 for Finger 1, 0.819 for Finger 2, 0.822 for Finger 3, and 0.668 for Finger 4, demonstrating the model's ability to capture finger flexion dynamics effectively. Validation performance showed a decline, with correlations of 0.700 for Finger 0, 0.325 for Finger 1, 0.438 for Finger 2, 0.447 for Finger 3, and 0.426 for Finger 4, indicating some overfitting to the training data. The test set results for Patient 2 were 0.618 for Finger 0, 0.370 for Finger 1, 0.378 for Finger 2, 0.552 for Finger 3, and 0.389 for Finger 4, maintaining moderate predictive accuracy despite the generalization challenge.

For Patient 3, the training set correlations were 0.731 for Finger 0, 0.714 for Finger 1, 0.581 for Finger 2, 0.728 for Finger 3, and 0.591 for Finger 4, reflecting consistent learning across fingers. Validation correlations improved slightly for some fingers, with 0.600 for Finger 0, 0.384 for Finger 1, 0.441 for Finger 2, 0.465 for Finger 3, and 0.463 for Finger 4, suggesting better generalization compared to Patient 2. The test set performance was strong, with correlations of 0.799 for Finger 0, 0.590 for Finger 1, 0.680 for Finger 2, 0.706 for Finger 3, and 0.573 for Finger 4, indicating robust predictive capability on unseen data.

The predicted finger flexion trajectories generally follow the trends of the actual data, particularly during significant flexion events, though the model tends to produce noisier predictions. Overall, the algorithm demonstrates promising performance for brain-computer interface applications, with variations across patients and fingers underscoring the need for further refinement to enhance generalization and reduce noise in predictions.

The training and testing plots for all the patients are attached below in the Appendix, after the main report. Below is a validation plot for Patient 1, which motivated us to reinforce and fine-tune our algorithm parameters.

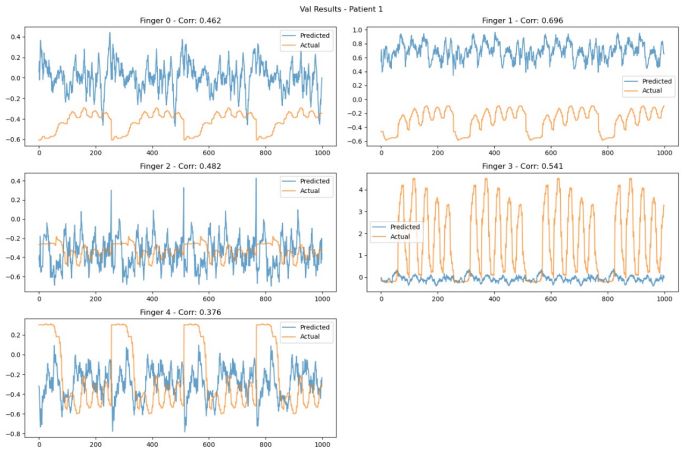

Fig. 3. Validation Plots for Patient 1, reinforcing our model architecture

V. Discussion & Conclusion

The final project was a really nice experience for us as a team. Despite the trials, tribulations, and late nights this project presented, we thoroughly enjoyed the process. While we are incredibly proud of our performance, the true prizes of the competition are the skills we have gained and refined over the course of the past few weeks, as well as the opportunity to meet, work with, and exchange knowledge with fellow students with unique backgrounds and shared interests. Through this experience, we have grown as communicators and gained a newfound appreciation for the interdisciplinary approaches that propel BCIs forward and give those fortunate enough to access them a new lease on life.

We found a great sense of accomplishment in passing each of the checkpoints and working our way up the leaderboard. That being said, there is still a great deal for us to learn. As the final days of the project closed in, we came across alternative approaches that we were not fully sure how to implement, and others that had the potential to improve our final correlation, but could not be feasibly implemented before the deadline for final algorithm submission. For instance, Flamary and Rakotomamonjy (2012) utilized a switching linear model, but used Auto-Regressive coefficients during feature extraction, which captured signal properties differently from our chosen time-frequency analysis via continuous wavelet transform (CWT).

Meanwhile, Dubey and Ray (2020) applied a similar approach to bandpass filtering, but with different Butterworth filters at varying frequency ranges; we lacked an understanding of how to identify the right order to use for given frequencies, nor were we certain how transferrable the study, which involved parsing gamma bands from LFP in the visual cortex, would be to finger movements.

When we applied filtering to reduce the noise in the dataset, our correlation coefficients were lower compared to attempts with little to no filtering. This may be due in part to our choice of notch filtering - removing power line interference at 60 Hz may have removed informative signaling from the upper bound of low-gamma bands and the lower bound of high-gamma bands. Furthermore, IIR filters introduce phase shifts that could have complicated our time-frequency analysis, and applying multiple harmonics may have filtered out spectral content with predictive value. Sharma and Sharma (2022) used Variational Mode Decomposition (VMD) and avenues of feature extraction involving correntropy, cross informational potential, and Kozachenko-Leonenko entropy. While the prospect of introducing VMD and entropy seemed like an effective solution to filtering out background noise, this would have necessitated a reworking of our pipeline. We later found that Shi, Yu, and Li (2021) used a similar approach to CWT that extended each section of signals symmetrically on both sides, a feasible addition to our filtering stage that could have aided in reducing distortion of the spectrum.

In conclusion, the developed algorithm, leveraging a 1D convolutional autoencoder within a PyTorch Lightning framework, successfully predicts finger flexion from ECoG signals, achieving an impressive average correlation coefficient of 0.5455 across three subjects, surpassing the BE 521 Final Project Competition checkpoints. By integrating wavelet-based feature extraction, meticulous preprocessing to align neural and motor data, and a robust encoder-decoder architecture with skip connections, it effectively captures the complex dynamics of motor intentions, demonstrating significant potential for brain-computer interface applications. While the pipeline's modular design and comprehensive evaluation facilitate practical deployment, opportunities for further optimization through hyperparameter tuning and extended training could enhance its performance, paving the way for neural decoding solutions.

On a larger scale, several of the approaches we combined were adapted from our literature review. However, making micro-adjustments to parameters (e.g. changing the moving window size for time-frequency analysis) could have incrementally improved our final correlation. We could have narrowed heuristics by taking an approach analogous to 'Occam's razor,' wherein the simplest measures can be effective. Di Sapia et al. (2023) distinguished between background activity, spikes, and sharp waves in a more rudimentary way that still performed well in identifying temporal variation of ECoG signal dimension, albeit in the context of epileptogenesis in TBI.

Moreover, this project could have immensely benefited from the clinical input of healthcare professionals intimately familiar with reading and interpreting ECoG data, as their insights and intuition may not readily translate into any specific quantitative filtering methodology. We imagine that this could entail adapting the dataset so that it can be loaded into software like Clampfit and having someone with said expertise manually sift through the data and explain their observations. Collectively, encountering and reflecting on these challenges and shortcomings has made us even more motivated to continue honing our abilities so that we can improve our final algorithm and better equip ourselves to take on the future obstacles and advancements in the realm of BCIs.

References

1) Bougrain, L., Liang, N. (2009). Band-specific features improve Finger Flexion Prediction from ECoG.
2) Di Sapia, R., Rizzi, M., Moro, F., Lisi, I., Caccamo, A., Ravizza, T., Vezzani, A., Zanier, E. R. (2023). ECoG spiking activity and signal dimension are early predictive measures of epileptogenesis in a translational mouse model of traumatic brain injury. *Neurobiology of Disease*, *185*, Article 106251. https://doi.org/10.1016/j.nbd.2023.106251
3) Dubey, A., Ray, S. Comparison of tuning properties of gamma and high-gamma power in local field potential (LFP) versus electrocorticogram (ECoG) in visual cortex. *Sci Rep* 10, 5422 (2020). https://doi.org/10.1038/s41598-020-61961-9
4) Elghrabawy, A., Wahed, M. (2012). Prediction of five-class finger flexion using ECoG signals. *2012 Cairo International Biomedical Engineering Conference (CIBEC)*, 1-5. https://doi.org/10.1109/CIBEC.2012.6473300.
5) Flamary, R., Rakotomamonjy, A. (2012). Decoding finger movements from ECoG signals using switching

linear models. *Frontiers in Neuroscience*, *6*, Article 29. https://doi.org/10.3389/fnins.2012.00029

6) Kubanek, J., Miller, K. J., Ojemann, J. G., Wolpaw, J. R., & Schalk, G. (2009). Decoding flexion of individual fingers using electrocorticographic signals in humans. *Journal of Neural Engineering*, *6*(6), Article 066001. https://doi.org/10.1088/1741-2560/6/6/066001
7) Lung, B. E., Burns, B. (2023). Anatomy, Shoulder and Upper Limb, Hand Flexor Digitorum Profundus Muscle. In *StatPearls*. StatPearls Publishing.
8) Merino, E., Faes, A., Van Hulle, M. (2023). The role of distinct ECoG frequency features in decoding finger movement. *Journal of Neural Engineering*, 20. https://doi.org/10.1088/1741-2552/ad0c5e.
9) Miller, K. J., Schalk, G. (2008). *Prediction of finger flexion: 4th Brain-Computer Interface Data Competition* (Competition description). Retrieved from https://www.bbci.de/competition/iv/
10) R, K., Chowdhury, E., Mahadevappa, M., Kumar, C. (2024). Identification of Finger Movement from ECoG Signal Using Machine Learning Model. *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, 1-6. https://doi.org/10.1109/I2CT61223.2024.10543868.
11) Schalk, G., Kubanek, J., Miller, K. J., Anderson, N. R., Leuthardt, E. C., Ojemann, J. G., Limbrick, D., Moran, D. W., Gerhardt, L. A., Wolpaw, J. R. (2007). Decoding two-dimensional movement trajectories using electrocorticographic signals in humans. *Journal of Neural Engineering*, *4*(3), 264–275. https://doi.org/10.1088/1741-2560/4/3/012
12) Sharma, S., Sharma, R. R. (2022). Variational mode decomposition-based finger flexion detection using ECoG signals. In V. Bajaj G. R. Sinha (Eds.), *Artificial intelligence-based brain-computer interface* (pp. 261–282). Academic Press. https://doi.org/10.1016/B978-0-323-91197-9.00003-5
13) Shi, H., Yu, P., Li, H. (2021). The finger flexion related feature extraction method based on wavelet time-frequency analysis in ECoG signals. In *Proceedings of the 5th International Conference on Computer Science and Application Engineering* (Article 24). Association for Computing Machinery. https://doi.org/10.1145/3487075.3487099
14) Warland, D. K., Reinagel, P., & Meister, M. (1997). Decoding visual information from a population of retinal ganglion cells. *Journal of Neurophysiology*, *78*(5), 2336–2350. https://doi.org/10.1152/jn.1997.78.5.2336
15) Xie, Z., Schwartz, O., Prasad, A. (2018). Decoding of finger trajectory from ECoG using deep learning. *Journal of Neural Engineering*, 15. https://doi.org/10.1088/1741-2552/aa9dbe.

VI. Appendix - I

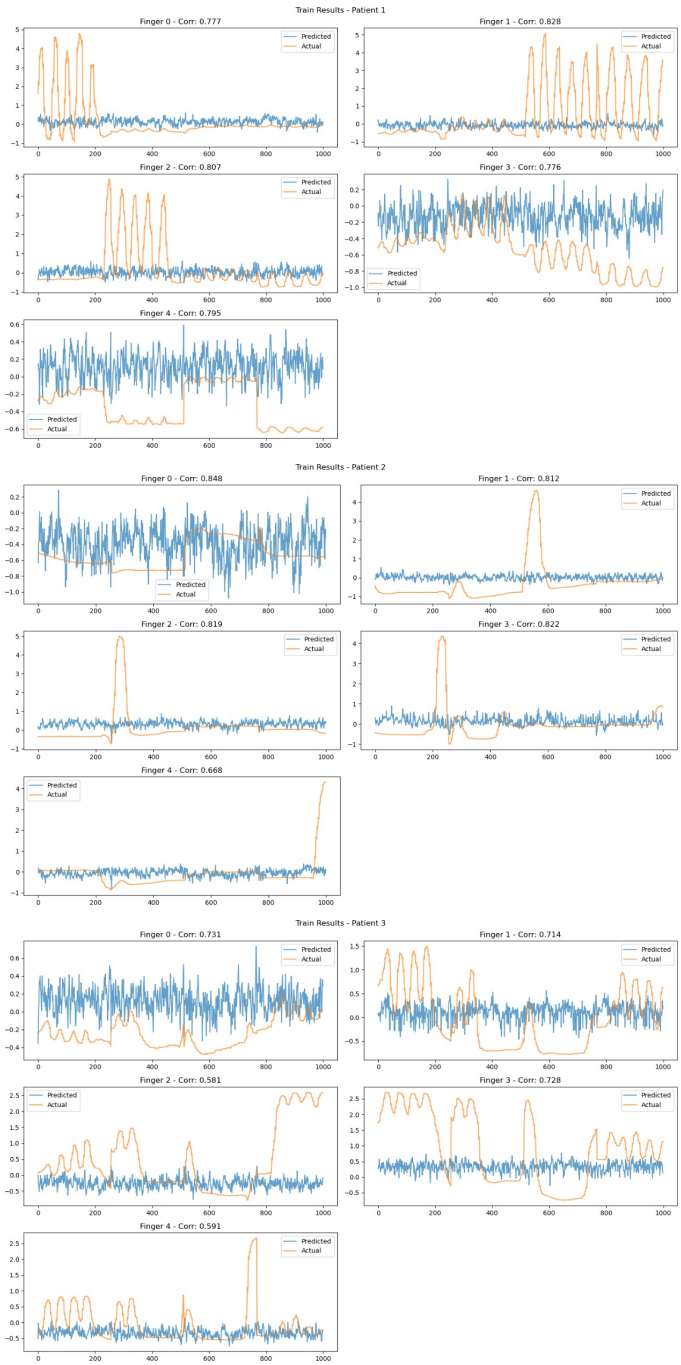

Fig. 4. Training Results for Patients 1, 2, 3 for all 5 Fingers.

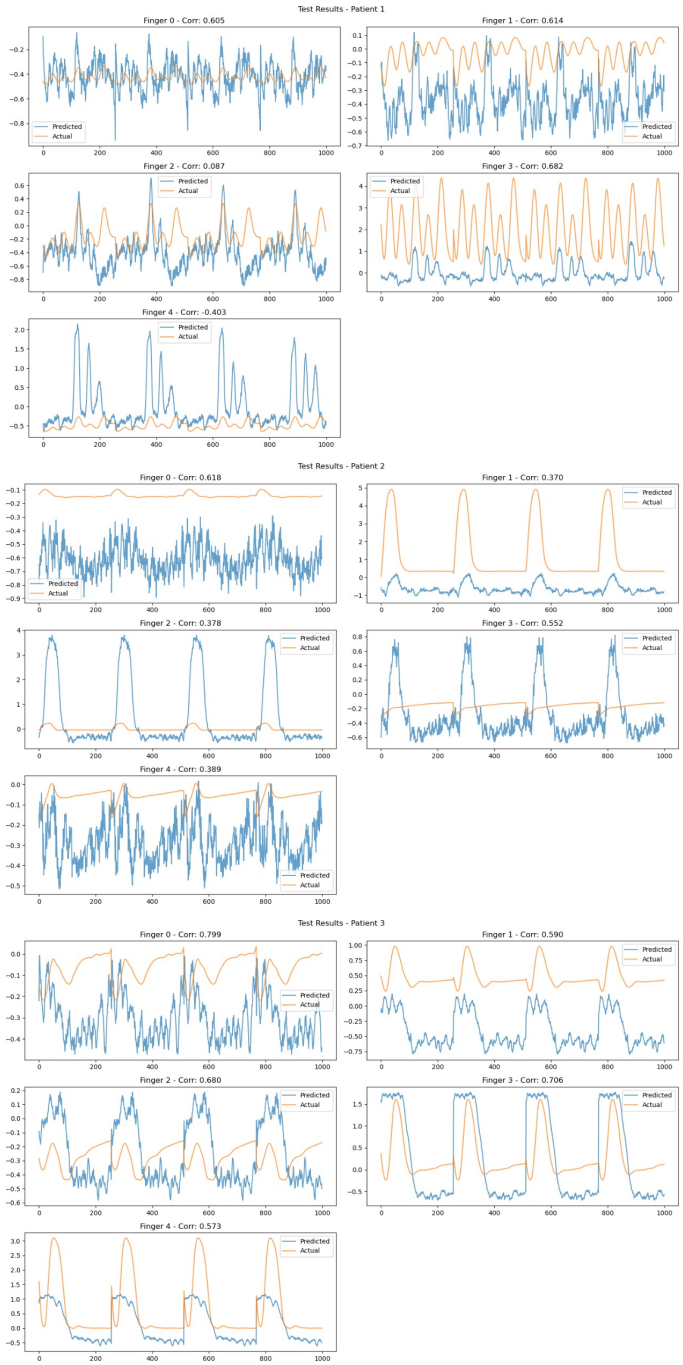

Fig. 5. Test Results for Patients 1, 2, 3 for all 5 Fingers.

VII. Appendix - II

```python
import os
import random
import time
import datetime
import numpy as np
import scipy.signal
import scipy.ndimage
import pywt
import pathlib
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import pytorch_lightning as pl
from pytorch_lightning import Trainer
from pytorch_lightning.callbacks import Callback, ModelCheckpoint, EarlyStopping
import scipy.io
from tqdm import tqdm

torch.set_float32_matmul_precision('high')

# Set random seeds
seed = 42
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
np.random.seed(seed)
random.seed(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Hyperparameters
L_FREQ, H_FREQ = 40, 300
WAVELET_NUM = 40
DOWNSAMPLE_FS = 100
time_delay_secs = 0.137
SAMPLE_LEN = 256
finger_num = 5
ORIGINAL_FS = 1000
BATCH_SIZE = 64
CHANNELS_NUM = [62, 48, 64]
MAX_EPOCHS = 5
NUM_WORKERS = 4  # Increased for better data loading

# Paths
SAVE_PATH = os.path.join(pathlib.Path().resolve(), "data")
CHECKPOINT_DIR = os.path.join(pathlib.Path().resolve(), "checkpoints")
RES_NPY_DIR = os.path.join(pathlib.Path().resolve(), "results")
os.makedirs(os.path.join(SAVE_PATH, "train"), exist_ok=True)
os.makedirs(os.path.join(SAVE_PATH, "val"), exist_ok=True)
```

```
os.makedirs(os.path.join(SAVE_PATH, "test"), exist_ok=True)
os.makedirs(CHECKPOINT_DIR, exist_ok=True)
os.makedirs(RES_NPY_DIR, exist_ok=True)

def preprocess_ecog(ecog_data, original_fs=ORIGINAL_FS, target_fs=DOWNSAMPLE_FS):
    print("␣␣Applying␣bandpass␣filter␣(5-20␣Hz)...")
    sos = scipy.signal.butter(4, [L_FREQ, H_FREQ], btype='band', fs=original_fs,
        output='sos')
    ecog_filtered = scipy.signal.sosfiltfilt(sos, ecog_data, axis=0)
    print("␣␣Performing␣continuous␣wavelet␣transform...")
    scales = pywt.scale2frequency('cmor1.5-1.0', np.arange(1, WAVELET_NUM + 1)) *
        original_fs
    scales = scales[(scales >= L_FREQ) & (scales <= H_FREQ)]
    ecog_wavelet = np.zeros((ecog_data.shape[1], WAVELET_NUM, ecog_data.shape[0]))
    for ch in range(ecog_data.shape[1]):
        coef, _ = pywt.cwt(ecog_filtered[:, ch], scales, 'cmor1.5-1.0',
            sampling_period=1 / original_fs)
        ecog_wavelet[ch, :coef.shape[0], :] = np.abs(coef)
    ecog_wavelet = ecog_wavelet[:, :WAVELET_NUM, :]
    print("␣␣Downsampling␣to␣100␣Hz...")
    downsample_factor = int(original_fs // target_fs)
    ecog_downsampled = scipy.signal.decimate(ecog_wavelet, downsample_factor, axis
        =-1)
    if np.any(np.isnan(ecog_downsampled)) or np.any(np.isinf(ecog_downsampled)):
        raise ValueError("NaNs␣or␣Infs␣detected␣in␣preprocessed␣ECoG")
    return ecog_downsampled.astype(np.float32)

def preprocess_fingerflex(fingerflex_data, original_fs=ORIGINAL_FS, target_fs=
    DOWNSAMPLE_FS):
    print("␣␣Downsampling␣fingerflex␣data...")
    downsample_factor = int(original_fs // target_fs)
    fingerflex_downsampled = scipy.signal.decimate(fingerflex_data, downsample_factor
        , axis=0)
    if np.any(np.isnan(fingerflex_downsampled)) or np.any(np.isinf(
        fingerflex_downsampled)):
        raise ValueError("NaNs␣or␣Infs␣detected␣in␣preprocessed␣fingerflex")
    return fingerflex_downsampled.astype(np.float32)

def save_preprocessed_data(data):
    print("Preprocessing␣data␣for␣all␣patients...")
    start_time = time.time()
    for p in range(3):
        print(f"Processing␣Patient␣{p␣+␣1}...")
        ecog = data['train_ecog'][p, 0]
        dg = data['train_dg'][p, 0]
        ecog_proc = preprocess_ecog(ecog)
        dg_proc = preprocess_fingerflex(dg)
        print(f"Patient␣{p␣+␣1}␣ECoG␣shape:␣{ecog_proc.shape},␣Fingerflex␣shape:␣{
            dg_proc.shape}")

        # Applying time delay to fingerflex data
        print("␣␣Applying␣time␣delay␣to␣fingerflex␣data...")
```

```
        delay_samples = int(time_delay_secs * DOWNSAMPLE_FS)  # 0.137 * 100 = 13.7 ->
             14 samples
        print(f"␣␣Shifting␣fingerflex␣data␣by␣{delay_samples}␣samples␣(forward)...")
        dg_proc = np.roll(dg_proc, shift=-delay_samples, axis=0)  # Shift forward
        if delay_samples > 0:
            dg_proc[-delay_samples:, :] = 0  # Zero-pad the end

        # Ensure lengths match after shifting
        total_time = ecog_proc.shape[-1]
        if total_time != dg_proc.shape[0]:
            min_length = min(total_time, dg_proc.shape[0])
            ecog_proc = ecog_proc[..., :min_length]
            dg_proc = dg_proc[:min_length, :]
            total_time = min_length
        print(f"␣␣Aligned␣ECoG␣shape:␣{ecog_proc.shape},␣Fingerflex␣shape:␣{dg_proc.
            shape}")

        # Split into train, val, and test
        train_idx = int(0.7 * total_time)
        val_idx = int(0.85 * total_time)
        ecog_train = ecog_proc[..., :train_idx]
        ecog_val = ecog_proc[..., train_idx:val_idx]
        ecog_test = ecog_proc[..., val_idx:total_time]
        dg_train = dg_proc[:train_idx, :]
        dg_val = dg_proc[train_idx:val_idx, :]
        dg_test = dg_proc[val_idx:total_time, :]  # Ensure test split matches
            total_time

        # Save the preprocessed data
        train_ecog_path = os.path.join(SAVE_PATH, "train", f"ecog_data_p{p␣+␣1}.npy")
        np.save(train_ecog_path, ecog_train)
        np.save(os.path.join(SAVE_PATH, "train", f"fingerflex_data_p{p␣+␣1}.npy"),
            dg_train)
        np.save(os.path.join(SAVE_PATH, "val", f"ecog_data_p{p␣+␣1}.npy"), ecog_val)
        np.save(os.path.join(SAVE_PATH, "val", f"fingerflex_data_p{p␣+␣1}.npy"),
            dg_val)
        np.save(os.path.join(SAVE_PATH, "test", f"ecog_data_p{p␣+␣1}.npy"), ecog_test
            )
        np.save(os.path.join(SAVE_PATH, "test", f"fingerflex_data_p{p␣+␣1}.npy"),
            dg_test)
        if not os.path.exists(train_ecog_path):
            raise FileNotFoundError(f"Failed␣to␣save␣{train_ecog_path}")
        print(f"Saved␣preprocessed␣data␣for␣Patient␣{p␣+␣1}")
    print(f"Preprocessing␣took␣{time.time()␣-␣start_time:.2f}␣seconds")

class EcogFingerflexDataset(Dataset):
    def __init__(self, path_to_ecog_data: str, path_to_fingerflex_data: str,
        sample_len: int, train=False):
        if not os.path.exists(path_to_ecog_data):
            raise FileNotFoundError(f"ECoG␣data␣file␣not␣found:␣{path_to_ecog_data}")
        if not os.path.exists(path_to_fingerflex_data):
            raise FileNotFoundError(f"Fingerflex␣data␣file␣not␣found:␣{
                path_to_fingerflex_data}")
```

```
        self.ecog_data = np.load(path_to_ecog_data).astype(np.float32)
        self.fingerflex_data = np.load(path_to_fingerflex_data).astype(np.float32)
        if np.any(np.isnan(self.ecog_data)) or np.any(np.isinf(self.ecog_data)):
            raise ValueError("NaNs␣or␣Infs␣in␣ECoG␣data")
        if np.any(np.isnan(self.fingerflex_data)) or np.any(np.isinf(self.
            fingerflex_data)):
            raise ValueError("NaNs␣or␣Infs␣in␣fingerflex␣data")
        self.duration = self.ecog_data.shape[-1]
        self.sample_len = sample_len
        self.stride = 1
        self.ds_len = (self.duration - self.sample_len) // self.stride
        self.train = train

    def __len__(self):
        return self.ds_len

    def __getitem__(self, index):
        sample_start = index * self.stride
        sample_end = sample_start + self.sample_len
        ecog_sample = self.ecog_data[..., sample_start:sample_end]
        fingerflex_sample = self.fingerflex_data[sample_start:sample_end, :].
            transpose(1, 0)
        return torch.from_numpy(ecog_sample).float(), torch.from_numpy(
            fingerflex_sample).float()

class EcogFingerflexDatamodule(pl.LightningDataModule):
    def __init__(self, sample_len: int, data_dir=SAVE_PATH, batch_size=BATCH_SIZE,
        patient_id=1):
        super().__init__()
        self.data_dir = data_dir
        self.sample_len = sample_len
        self.batch_size = batch_size
        self.add_name = f"_p{patient_id}"
        self.test = None

    def setup(self, stage=None):
        print(f"Setting␣up␣datamodule␣for␣Patient␣{self.add_name[2:]}...")
        self.train = EcogFingerflexDataset(
            os.path.join(self.data_dir, "train", f"ecog_data{self.add_name}.npy"),
            os.path.join(self.data_dir, "train", f"fingerflex_data{self.add_name}.npy
                "),
            self.sample_len, train=True
        )
        self.val = EcogFingerflexDataset(
            os.path.join(self.data_dir, "val", f"ecog_data{self.add_name}.npy"),
            os.path.join(self.data_dir, "val", f"fingerflex_data{self.add_name}.npy")
                ,
            self.sample_len
        )
        if stage == "test" or stage is None:
            self.test = EcogFingerflexDataset(
                os.path.join(self.data_dir, "test", f"ecog_data{self.add_name}.npy"),
```

```
                os.path.join(self.data_dir, "test", f"fingerflex_data{self.add_name}.
                    npy"),
                self.sample_len
            )
        print("Datamodule␣setup␣complete.")

    def train_dataloader(self):
        return DataLoader(self.train, batch_size=self.batch_size, num_workers=
            NUM_WORKERS, persistent_workers=True, shuffle=True)

    def val_dataloader(self):
        return DataLoader(self.val, batch_size=self.batch_size, persistent_workers=
            True, num_workers=NUM_WORKERS)

    def test_dataloader(self):
        if self.test is None:
            self.setup(stage="test")
        return DataLoader(self.test, batch_size=self.batch_size, persistent_workers=
            True, num_workers=NUM_WORKERS)

class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1, dilation=1,
        p_conv_drop=0.1):
        super().__init__()
        self.conv1d = nn.Conv1d(in_channels, out_channels, kernel_size, bias=False,
            padding='same')
        self.norm = nn.LayerNorm(out_channels)
        self.activation = nn.GELU()
        self.drop = nn.Dropout(p=p_conv_drop)
        self.downsample = nn.MaxPool1d(kernel_size=stride, stride=stride)
        self.stride = stride

    def forward(self, x):
        x = self.conv1d(x)
        x = torch.transpose(x, -2, -1)
        x = self.norm(x)
        x = torch.transpose(x, -2, -1)
        x = self.activation(x)
        x = self.drop(x)
        x = self.downsample(x)
        return x

class UpConvBlock(nn.Module):
    def __init__(self, scale, **args):
        super().__init__()
        self.conv_block = ConvBlock(**args)
        self.upsample = nn.Upsample(scale_factor=scale, mode='linear', align_corners=
            False)

    def forward(self, x):
        x = self.conv_block(x)
        x = self.upsample(x)
        return x
```

```python
class AutoEncoder1D(nn.Module):
    def __init__(self, n_electrodes=64, n_freqs=40, n_channels_out=5,
                 channels=[32, 32, 64, 64, 128, 128], kernel_sizes=[7, 7, 5, 5, 5],
                 strides=[2, 2, 2, 2, 2], dilation=[1, 1, 1, 1, 1]):
        super().__init__()
        self.n_inp_features = n_freqs * n_electrodes
        self.model_depth = len(channels) - 1
        self.spatial_reduce = ConvBlock(self.n_inp_features, channels[0], kernel_size
            =3)
        self.downsample_blocks = nn.ModuleList([
            ConvBlock(channels[i], channels[i + 1], kernel_sizes[i], strides[i],
                dilation[i])
            for i in range(self.model_depth)
        ])
        channels = channels[:-1] + channels[-1:]
        self.upsample_blocks = nn.ModuleList([
            UpConvBlock(scale=strides[i],
                        in_channels=channels[i + 1] if i == self.model_depth - 1 else
                             channels[i + 1] * 2,
                        out_channels=channels[i], kernel_size=kernel_sizes[i])
            for i in range(self.model_depth - 1, -1, -1)
        ])
        self.conv1x1_one = nn.Conv1d(channels[0] * 2, n_channels_out, kernel_size=1,
            padding='same')

    def forward(self, x):
        batch, elec, n_freq, time = x.shape
        x = x.reshape(batch, -1, time)
        x = self.spatial_reduce(x)
        skip_connection = []
        for i in range(self.model_depth):
            skip_connection.append(x)
            x = self.downsample_blocks[i](x)
        for i in range(self.model_depth):
            x = self.upsample_blocks[i](x)
            x = torch.cat((x, skip_connection[-1 - i]), dim=1)
        x = self.conv1x1_one(x)
        return x

class BaseEcogFingerflexModel(pl.LightningModule):
    def __init__(self, model):
        super().__init__()
        self.model = model
        self.lr = 8.42e-5
        self.all_metrics = {
            'train': {f'finger_{i}': [] for i in range(finger_num)},
            'val': {f'finger_{i}': [] for i in range(finger_num)},
            'test': {f'finger_{i}': [] for i in range(finger_num)}
        }

    def forward(self, x):
        return self.model(x)
```

```
    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self.model(x)
        loss = F.mse_loss(y_hat, y)
        corr = correlation_metric(y_hat, y)
        self.log("train_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        self.log("train_corr", corr, on_step=False, on_epoch=True, prog_bar=True)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self.model(x)
        loss = F.mse_loss(y_hat, y)
        corr = correlation_metric(y_hat, y)
        self.log("val_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        self.log("val_corr", corr, on_step=False, on_epoch=True, prog_bar=True)
        return {'val_loss': loss, 'val_corr': corr, 'preds': y_hat, 'targets': y}

    def test_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self.model(x)
        loss = F.mse_loss(y_hat, y)
        corr = correlation_metric(y_hat, y)
        self.log("test_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        self.log("test_corr", corr, on_step=False, on_epoch=True, prog_bar=True)
        return {'test_loss': loss, 'test_corr': corr, 'preds': y_hat, 'targets': y}

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.lr, weight_decay=1e-6)

class ComprehensiveEvaluationCallback(Callback):
    def __init__(self, patient_id):
        super().__init__()
        self.patient_id = patient_id
        self.results = {
            'train': {'predictions': [], 'targets': []},
            'val': {'predictions': [], 'targets': []},
            'test': {'predictions': [], 'targets': []}
        }

    def on_train_batch_end(self, trainer, pl_module, outputs, batch, batch_idx):
        x, y = batch
        with torch.no_grad():
            y_hat = pl_module.model(x)
        self.results['train']['predictions'].append(y_hat.cpu().numpy())
        self.results['train']['targets'].append(y.cpu().numpy())

    def on_validation_batch_end(self, trainer, pl_module, outputs, batch, batch_idx):
        x, y = batch
        with torch.no_grad():
            y_hat = pl_module.model(x)
        self.results['val']['predictions'].append(y_hat.cpu().numpy())
```

```
        self.results['val']['targets'].append(y.cpu().numpy())

    def on_test_batch_end(self, trainer, pl_module, outputs, batch, batch_idx):
        x, y = batch
        with torch.no_grad():
            y_hat = pl_module.model(x)
        self.results['test']['predictions'].append(y_hat.cpu().numpy())
        self.results['test']['targets'].append(y.cpu().numpy())

    def on_train_end(self, trainer, pl_module):
        self._process_and_save_results('train', pl_module)

    def on_validation_end(self, trainer, pl_module):
        self._process_and_save_results('val', pl_module)

    def on_test_end(self, trainer, pl_module):
        self._process_and_save_results('test', pl_module)

    def _process_and_save_results(self, dataset_type, pl_module):
        preds = np.concatenate(self.results[dataset_type]['predictions'], axis=0)
        targets = np.concatenate(self.results[dataset_type]['targets'], axis=0)
        correlations = []
        for finger in range(finger_num):
            pred_finger = preds[:, finger, :].flatten()
            target_finger = targets[:, finger, :].flatten()
            corr = corr_metric(pred_finger, target_finger)
            correlations.append(corr)
            pl_module.all_metrics[dataset_type][f'finger_{finger}'].append(corr)
        np.save(os.path.join(RES_NPY_DIR, f"{dataset_type}_predictions_p{self.
            patient_id}.npy"), preds)
        np.save(os.path.join(RES_NPY_DIR, f"{dataset_type}_targets_p{self.patient_id
            }.npy"), targets)
        self._plot_results(dataset_type, preds, targets, correlations)

    def _plot_results(self, dataset_type, preds, targets, correlations):
        plt.figure(figsize=(15, 10))
        for finger in range(finger_num):
            plt.subplot(3, 2, finger + 1)
            pred_finger = preds[:, finger, :].flatten()
            target_finger = targets[:, finger, :].flatten()
            plt.plot(pred_finger[:1000], label='Predicted', alpha=0.7)
            plt.plot(target_finger[:1000], label='Actual', alpha=0.7)
            plt.title(f"Finger␣{finger}␣-␣Corr:␣{correlations[finger]:.3f}")
            plt.legend()
        plt.suptitle(f"{dataset_type.capitalize()}␣Results␣-␣Patient␣{self.patient_id
            }")
        plt.tight_layout()
        plt.savefig(os.path.join(RES_NPY_DIR, f"{dataset_type}_results_p{self.
            patient_id}.png"))
        plt.close()

def correlation_metric(x, y):
    x = x.flatten()
```

```
    y = y.flatten()
    vx = x - torch.mean(x)
    vy = y - torch.mean(y)
    corr = torch.sum(vx * vy) / (torch.sqrt(torch.sum(vx ** 2)) * torch.sqrt(torch.
        sum(vy ** 2)))
    return torch.clamp(corr, -1.0, 1.0)  # Prevent NaNs

def corr_metric(x, y):
    x, y = x.flatten(), y.flatten()
    if np.std(x) < 1e-6 or np.std(y) < 1e-6:
        return 0.0
    corr = np.corrcoef(x, y)[0, 1]
    return 0.0 if np.isnan(corr) else corr  # Handle NaN correlations

def plot_final_comparison(all_patient_results):
    plt.figure(figsize=(15, 10))
    for dataset_idx, dataset_type in enumerate(['train', 'val', 'test']):
        for patient_id in range(1, 4):
            plt.subplot(3, 3, dataset_idx * 3 + patient_id)
            patient_key = f"patient_{patient_id}"
            metrics = all_patient_results[patient_key][dataset_type]
            finger_corrs = []
            for finger in range(finger_num):
                finger_key = f"finger_{finger}"
                if metrics[finger_key]:
                    finger_corrs.append(np.mean(metrics[finger_key]))
                else:
                    finger_corrs.append(0)
            plt.bar(range(finger_num), finger_corrs)
            plt.xticks(range(finger_num), [f'Finger␣{i}' for i in range(finger_num)])
            plt.ylim(0, 1)
            plt.title(f"Patient␣{patient_id}␣-␣{dataset_type.capitalize()}")
            plt.ylabel("Mean␣Correlation")
    plt.suptitle("Final␣Performance␣Comparison␣Across␣Patients")
    plt.tight_layout()
    plt.savefig(os.path.join(RES_NPY_DIR, "final_comparison.png"))
    plt.close()

def main():
    print(f"Starting␣BCI␣training␣pipeline␣at␣{datetime.datetime.now()}")
    print(f"SAVE_PATH:␣{SAVE_PATH}")
    print(f"Expected␣train␣file:␣{os.path.join(SAVE_PATH,␣'train',␣'ecog_data_p1.npy
        ')}")
    start_time = time.time()
    if not torch.cuda.is_available():
        print("Error:␣CUDA␣not␣available.␣Please␣check␣GPU␣drivers␣and␣PyTorch␣
            installation.")
        return
    print(f"GPU␣detected:␣{torch.cuda.get_device_name(0)}")
    torch.cuda.empty_cache()
    import gc
    gc.collect()
```

```
try:
    data = scipy.io.loadmat("C:\\Users\\dhyey\\OneDrive\\Desktop\\MSE_ROBO\\Sem_2
        \\Brain_Computer_Interfaces\\Assignments\\Final_Project\\
        raw_training_data.mat")
    if 'train_ecog' not in data or 'train_dg' not in data:
        raise KeyError("Dataset␣must␣contain␣'train_ecog'␣and␣'train_dg'␣keys")
except Exception as e:
    print(f"Error␣loading␣dataset:␣{e}")
    return

print("Preprocessing␣data...")
save_preprocessed_data(data)

all_patient_results = {}
for patient_id in range(1, 4):
    print(f"\n===␣Processing␣Patient␣{patient_id}␣===")
    hp_autoencoder = {
        "channels": [32, 32, 64, 64, 128, 128],
        "kernel_sizes": [7, 7, 5, 5, 5],
        "strides": [2, 2, 2, 2, 2],
        "dilation": [1, 1, 1, 1, 1],
        "n_electrodes": CHANNELS_NUM[patient_id - 1],
        "n_freqs": WAVELET_NUM,
        "n_channels_out": finger_num
    }
    model = AutoEncoder1D(**hp_autoencoder).to("cuda")
    lightning_wrapper = BaseEcogFingerflexModel(model)
    dm = EcogFingerflexDatamodule(sample_len=SAMPLE_LEN, patient_id=patient_id)
    dm.setup(stage="fit")
    checkpoint_callback = ModelCheckpoint(
        save_top_k=2,
        monitor="val_corr",
        mode="max",
        dirpath=CHECKPOINT_DIR,
        filename=f"model_p{patient_id}-{{epoch:02d}}-{{val_corr:.4f}}"
    )
    early_stop_callback = EarlyStopping(
        monitor="val_corr",
        min_delta=0.001,
        patience=5,
        verbose=True,
        mode="max"
    )
    eval_callback = ComprehensiveEvaluationCallback(patient_id)
    trainer = Trainer(
        accelerator="gpu",
        devices=1,
        max_epochs=MAX_EPOCHS,
        callbacks=[checkpoint_callback, early_stop_callback, eval_callback],
        logger=True,
        enable_progress_bar=True
    )
    print(f"\nTraining␣Patient␣{patient_id}␣model...")
```

```
        trainer.fit(lightning_wrapper, dm)
        print(f"\nTesting␣Patient␣{patient_id}␣model...")
        try:
            dm.setup(stage="test")
            test_loader = dm.test_dataloader()
            if len(test_loader.dataset) == 0:
                raise ValueError("Test␣dataset␣is␣empty")
            trainer.test(lightning_wrapper, dataloaders=test_loader, ckpt_path="best"
                )
        except Exception as e:
            print(f"Error␣during␣testing␣for␣Patient␣{patient_id}:␣{e}")
            continue
        all_patient_results[f"patient_{patient_id}"] = lightning_wrapper.all_metrics
    print(f"\nTotal␣run␣time:␣{time.time()␣-␣start_time:.2f}␣seconds")
    print("\nGenerating␣final␣comparison␣plots...")
    plot_final_comparison(all_patient_results)
    print("\nTraining␣and␣evaluation␣complete!")

if __name__ == "__main__":
    main()
```